

The Living Map — Phase 1 Project Status & Technical Handover

TSYP14 Technical Challenge · IEEE RAS × IEEE AESS, Tunisia Section Chapters

Checkpoint written on **03/10/2026** · Official Phase 1 submission deadline: **05/10/2026**

Purpose	Let a new reader (the user) and a new AI agent understand the whole project at once, without losing the work already done.
Primary sources	The two official documents (cahier des charges, 3 pages; presentation, 8 slides) and the work done so far with the AI co-pilot. Nothing in this document comes from outside these sources, except hardware facts that were looked up on the web and are marked as such.
Project owner / role	The user is new to the project and is building the Writer subsystem. The split of the team into four roles (Writer, Beacon, Outside Network, Executor) was proposed by the AI; the names of the other three teammates are not known to the AI.
Companion file	writer_sim_snapshot_2026-10-03.zip — the Python simulation code exactly as it is today (work in progress, with known bugs, see section 6).

The project in 8 lines

- The challenge asks for a **two-robot system** (Writer + Executor) and an **Outside Network Area** that together give an unmapped, GPS-denied space a memory, stored in radio beacons.
- The challenge is understood and a concept is designed (section 3). The Writer hardware has been chosen by the user (section 4).
- A **Python simulation of the Writer** exists (section 5). It works in many normal runs but is **not yet reliable**: some runs skip a room, show a wrong “stuck” event, or wander too long (section 6).
- Only the Writer is simulated. **Beacon radio, Outside Network, frame translation, Executor and the command-post live map are designed only, not implemented.**
- No physical hardware has been tested. Nothing is physically validated.
- The failure scenarios exist as command-line options in the code but **have never been run** (section 9).
- Interfaces with the teammates (beacon format, log format, direction meaning) are **drafts, not agreed** (sections 7 and 8).
- Nothing is packaged for submission yet: no GitHub repository, no report, no final diagrams, no clean demo output (sections 10-12).

Labels used everywhere in this document

[CONFIRMED]	Written in the official documents, or stated explicitly by the user.
[PROPOSED]	Suggested by the AI during the project. The user has not explicitly approved it.
[ASSUMPTION]	A value or fact chosen to make the design or the simulation work. Must be validated (many only on real hardware).
[OPEN]	No decision yet.
[IMPLEMENTED IN SIM] / [DESIGN ONLY]	Exists as code in the simulation / exists only as an idea or text.
[TESTED] / [UNTESTED]	The code path has / has not actually been run and observed.
[INTERPRETATION]	The AI’s reading of the official documents (not written there).

1. Challenge overview

1.1 Name and organizers

“The Living Map: Spatial Memory for Emergency Robots” — TSYP14 Technical Challenge, organized by **IEEE RAS** (Robotics & Automation Society) and **IEEE AESS** (Aerospace and Electronic Systems Society), Tunisia Section Chapters. **[CONFIRMED]**

1.2 The problem, in simple words

In disasters (collapsed buildings, mines, fires, contaminated zones) robots may have to work where **there is no GPS and no communication network**. They must explore, sense dangers, find victims, and report back despite poor connectivity. The key weakness stated by the organizers: **when a robot fails, leaves, or loses power, everything it learned can disappear with it**. The challenge wants a system that can **Sense → Communicate → Preserve critical information → Enable mission continuity**.

1.3 The “Living Map” idea

A first robot enters an unmapped space alone and **leaves its knowledge behind in small radio beacons**. A second robot, entering later, **reads those beacons and continues the mission without starting from zero**. The space itself carries the memory — it does not depend on the first robot surviving. (The official scope text says this in other words.) **[CONFIRMED]**

1.4 What must be built

Part	Role (according to the official documents)
Writer robot	Enters the GPS-denied space first, alone, with no map and no operator. Explores autonomously, senses events (at least 2 types, e.g. hazard, victim, hole, gas, junction, exit), decides in real time which findings are worth recording, deposits a beacon where each event is, then returns to the entrance.
Beacons	Tiny ground transmitters that just broadcast a compact message saying <i>what</i> was found, <i>where to go</i> and <i>when</i> it was written. Messages must age (see below).
Outside Network Area	The bridge between inside and outside. Inside: no GPS, no network. Outside: GPS, satellite time and communication exist. Defined by four roles (teams design the inside of it): RECEIVE what the Writer produced; TRANSLATE the robot's private coordinates into real-world GPS; CARRY the information to a distant command post (wireless or satellite); BRIEF the Executor before it enters.
Executor robot	Enters later with a mission (extinguish, extract, seal or retrieve — tied to the chosen scenario). Briefed before entry, so no blind search. Navigates using the beacons as its map, no GPS. “Trusts by age”: fresh beacons guide it, stale ones it verifies with its own sensors.
Command post	Distant location that receives the information and displays a live map .

Beacon aging (from the slides): the same beacon is read differently as time passes. Slide examples: *fresh (2 min)* “trust as measured”; *aging (15 min)* “verify before crossing”; *stale (40 min)* “rely on your own senses”; *suspect (contradicted)* “live sensor disagrees: flag it, discard”. A beacon message has six items: identifier, WHAT, WHERE (direction to take from here), distance (metres to next beacon / target), WHEN (timestamp, which makes it age), version (lets a later agent overwrite it). **[CONFIRMED]**

1.5 The required communication chain

Writer explores → events detected & beacons deposited → Outside Network receives & translates data → wireless / satellite communication → command post → Executor receives mission → Executor enters & navigates using beacons.

1.6 Mandatory rules for every team

- Complete registration before the deadline; choose **one** environment: Urban Search & Rescue, Fire / Hazardous Building, Mines / Tunnels, or Nuclear / Contaminated Zone.
- Detect **at least 2 event types**; use **at least 2 physical robots** (Writer and Executor).
- Implement a **functional Outside Network Area**; **all** inside↔outside communication passes through it.
- **No direct link** between the robots and the command post.
- Prepare and demonstrate **failure cases** (slide 6).

2. Phase 1 requirements

2.1 Required deliverables (cahier des charges p.2 and slide 8)

#	Deliverable	Official detail	Points shown in the scoring
1	GitHub Repository	—	4
2	Simulation Demo	Format (video / live) not specified	8
3	Short Technical Report	maximum 6 pages	2
4	Implementation Plan	—	3
5	Failure Cases	—	3
6	Technical Solution & Architecture	Scored through “Technical solution diagrams” and the 7 technical areas below	5 (diagrams) + 7 areas

The solution must address 7 areas (each worth 5 points in the scoring list): Writer robot autonomy · Event detection & beacon deposition · Beacon message & signal design · Frame translation · Outside network architecture · Executor robot · System architecture & data flow. A further criterion, **Innovation, Originality & Creativity**, is worth 5 points.

2.2 Deadlines and logistics (cahier des charges p.3, quoted)

Challenge Infosession	14/09/2026
Phase 1 Submission Deadline	05/10/2026 (the document gives no time of day)
Final Submission Deadline	01/12/2026
Prizes	2 winning teams: 1st place 100 USD + certificate; 2nd place 50 USD + certificate
Contact	aess@ieee.tn or ras@ieee.tn (“for any inquiries or clarifications”)
Links inside the PDF (not yet opened)	Page 2, “System Architecture Diagram” button → https://drive.google.com/file/d/1qZa4IAYTphZSt2vkvdIQwV6-f3tDO-ov/view?usp=sharing Page 3, “Submit your project...” text → a Google Forms link (probably the submission form; to be verified): https://docs.google.com/forms/d/e/1FAIpQLSdq93a-_h-U7WJBD-WEemnm-mm0XX1aCt-l64BGmpNK2EXqtA/viewform?usp=dialog

[OPEN] Check the deadline. The user mentioned at the start that Phase 1 was “due today” (03/10/2026); the official document says 05/10/2026. Confirm the real date, time and submission method (the Google Forms link above is the likely route).

2.3 Full scoring grid (as printed — note the inconsistency)

Block	Items and points (as printed)
Initial Phase (heading says 45 pts)	GitHub 4 · Simulation demo 8 · Short technical report 2 · Innovation, originality & creativity 5 · Implementation plan 3 · Failure cases 3 = 25
Technical aspects of the solution	Writer autonomy 5 · Event detection & beacon deposition 5 · Beacon message & signal design 5 · Frame translation 5 · Outside network area 5 · Executor robot 5 · Technical solution diagrams 5 = 35
Final Phase (50 pts)	Pitching & Q&A 8 · Quality of proposed solutions 8 · Technologies used 5 · Physical prototype 15 · Project architecture 5 · User manual 4 · GitHub repository 5
Bonus (5 pts)	≥ 2 IEEE AESS members in the team: 2 · ≥ 2 RAS members: 2 · ≥ 1 YP member with both RAS and AESS memberships: 1
Pitch (final phase)	English, 5 min pitch + 2 min Q&A. Finalists provide: pitching & Q&A, complete physical prototype, proposed solution, technologies used, project architecture, user manual, GitHub repository.

[OPEN] Scoring inconsistency: the Initial Phase is labelled 45 points, but the items listed under it and under “Technical aspects” add up to 60 (25 + 35). The total of the whole grid is 100 only if the Initial Phase counts as 45. Ask the organizers; until then treat **every** listed item as important. Not specified anywhere: submission platform/format, which simulator to use, whether the demo is a video, the report layout.

3. Our proposed solution

Everything in this section is **[PROPOSED]** unless marked otherwise. The AI compared three solution levels (S1 simple “breadcrumb”, S2 “aging memory map”, S3 “mesh memory” with beacon-to-beacon relay and extra radio) and **recommended designing at S2 level while keeping the Phase 1 simulation as simple as S1**. The user then proceeded on that basis without a formal vote.

Item	Current concept	Status
Environment	Fire / Hazardous Building . Reason: its events map to cheap sensors (heat, missing floor, geometry) and the Executor mission “extinguish” can be demonstrated safely with proxies. Alternative discussed: Urban Search & Rescue.	[PROPOSED] — not formally confirmed
Writer robot	Wheeled robot, Arduino UNO Q + RPLIDAR; explores by right-hand wall following; detects hazard (heat), junction and hole; drops beacons by servo; returns to the entrance; uploads a log (section 4).	Hardware [CONFIRMED] by user; logic [IMPLEMENTED IN SIM]
Beacons	Small RF broadcasters left on the floor, each carrying a compact message (what / direction / distance / time / version). The Writer writes the message by radio at drop time into a blank beacon with a unique ID (“D3”).	[PROPOSED] · [OPEN] (D3)
Outside Network Area	Raspberry Pi gateway with GPS near the entrance + a server/MQTT + a web live map at the command post. Satellite link <i>simulated</i> in Phase 1. Roles: Receive, Translate, Carry, Brief; store-and-forward queue if the link drops.	[PROPOSED] [DESIGN ONLY]
Executor robot	State flow BRIEFED → ENTER → LISTEN → TRUST-CHECK → FOLLOW → ACT → EXIT. Trust rule by beacon age: fresh = follow; aging = slow down and verify; stale = use own sensors; contradicted = flag and discard. Missing beacon: fall back to last heading/distance, then local search. Demo mission: “reach the hotspot and extinguish” (simulated action).	[PROPOSED] [DESIGN ONLY]

3.1 How information survives if the Writer cannot continue

Knowledge is stored **twice**. (1) **In the beacons**, which stay on the ground with their messages even if the Writer dies. (2) **In the Writer’s log**, which reaches the Outside Network only if the Writer comes back to the entrance. If the Writer is lost, the beacons remain as the only record; the Outside Network then knows nothing from the log, and the Executor can still read the beacons it meets. The simulation has an option to kill the Writer at a chosen time to show exactly this (**[UNTESTED]**).

3.2 How the Executor uses inherited information

Before entry, the Outside Network briefs the Executor (mission, ordered beacon chain, clock synchronisation, trust rules — **[PROPOSED]**). Inside, the Executor listens to beacons, judges each one by its age and by its own sensors, and follows the chain to the target. It enters through the same entrance, so it can share the Writer’s **private frame** (origin = entrance, +x = direction of entry). How the Executor keeps its heading consistent is **[OPEN]**.

3.3 System architecture

Writer explores -> beacons deposited -> Outside Network receives & translates -> command post -> Executor navigates by beacons

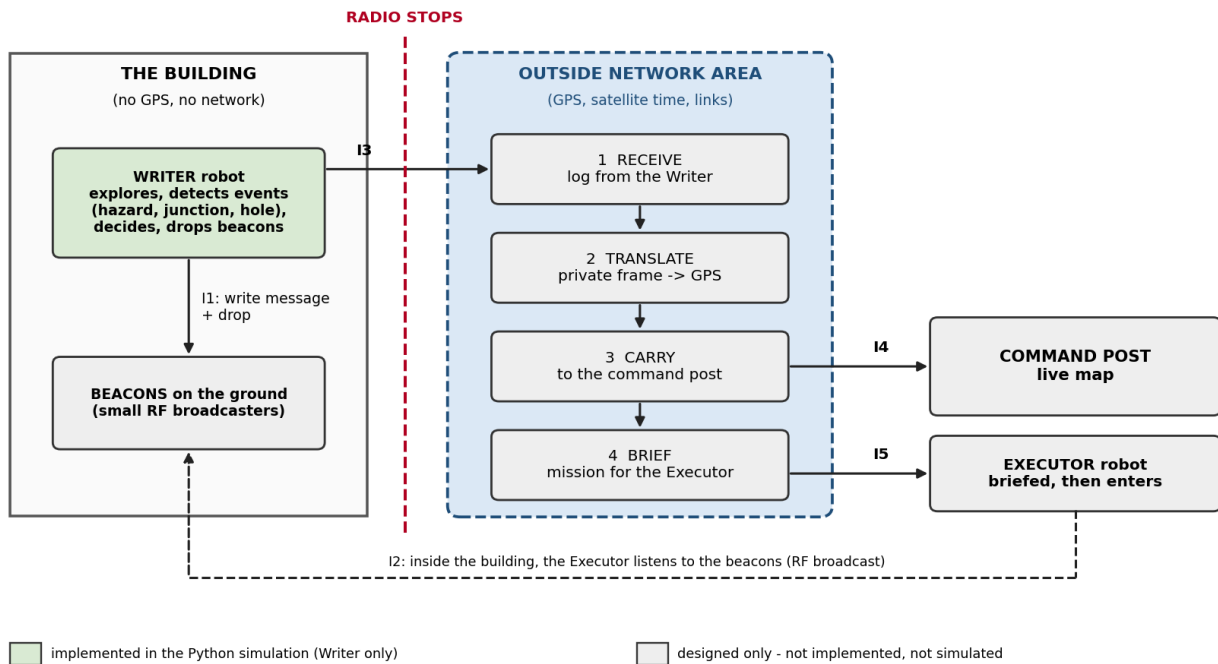


Figure 1 — Draft system architecture. Green = exists in the Python simulation; grey = designed only. Interfaces: I1 Writer→Beacon (message to write + drop); I2 Beacon→Executor (RF broadcast); I3 Writer→Outside Network (log, when the Writer returns); I4 Outside Network→Command post (GPS-referenced data for the live map); I5 Outside Network→Executor (briefing). All five are **[PROPOSED]**, none is agreed with the teammates.

3.4 Frame translation (private coordinates → GPS)

[PROPOSED] [DESIGN ONLY] The Writer works in a **private frame**: origin at the entrance, x axis = direction of entry, y = left. The Outside Network has a GPS fix of the entrance and needs the building's orientation (e.g. a compass heading taken outside — how to get it is not decided). It rotates each (x, y) by that heading to get East/North metres (E, N), then $lat = lat0 + N / 111\ 320$ and $lon = lon0 + E / (111\ 320 \cdot \cos lat0)$. Error budget: dead-reckoning drift grows with distance, so far beacons have less certain GPS positions, while beacon-to-beacon guidance stays accurate locally. **[ASSUMPTION]** Writer and Executor clocks are synchronised to GPS time before entry (needed for beacon timestamps and aging).

4. Writer subsystem

4.1 Hardware

Component	Role in the Writer	Status
Arduino “Q” (understood as Arduino UNO Q)	Main computer. Per Arduino documentation (web search, not verified on hardware): a Linux processor (Qualcomm QRB2210) <i>and</i> a real-time STM32U585 microcontroller on one board, linked by an RPC library called “Bridge”; 7–24 V DC input; USB peripherals need a powered USB-C dongle; processor I/O is 1.8 V. Proposed split: Linux side = “brain” (LiDAR, localization correction, exploration, event decisions, beacon policy, log, upload); microcontroller side = “reflexes” (motor PWM, encoders, IMU, servo, e-stop, watchdog).	Board: [CONFIRMED] by user (exact model [ASSUMPTION]); split: [PROPOSED]
RPLIDAR	Navigation: obstacle distance, wall following, junction detection, heading correction. Model not given by the user (A1-class assumed). Its USB adapter probably also needs the powered USB-C hub — test first .	[CONFIRMED] (model [ASSUMPTION])
Modulino Movement	Per documentation: LSM6DSOX 6-axis IMU (3-axis gyroscope + 3-axis accelerometer, <i>no compass</i>). Use: heading from integrating the gyro (with bias calibration at start), tip-over detection, jolt hint. Raw sensor: the software integrates the gyro itself. Do not use the accelerometer for distance.	[CONFIRMED] (user has it)
Modulino Distance	Per documentation: VL53L4CD time-of-flight sensor, 0–1200 mm, 3.3 V, Qwiic/I2C. Proposed role: floor-facing hole detector (gives the HOLE event). Alternatives: drop confirmation, or close-range front safety. One sensor = one role.	Owned: [CONFIRMED] ; role: [PROPOSED] [OPEN]
Servo + beacon magazine	Drops one beacon at a time. Concept: tube/stack magazine with a servo gate, plus an IR break-beam under the chute to confirm that a beacon fell. Servo model unknown; magazine not designed.	Servo: [CONFIRMED] ; mechanism: [PROPOSED]
Not yet chosen (unknown whether owned)	Drive base (two driven wheels + caster suggested; 4-wheel skid steer gives poor odometry), wheel encoders, motor driver, a thermal sensor for the heat hazard (MLX90640 or AMG8833 suggested; the Modulino Thermo only measures air temperature and is not suitable), beacon write radio, battery + separate 5 V rail for servo/LiDAR, e-stop button.	[PROPOSED] [OPEN]
Simulated in code	LiDAR (120 rays, 360°), thermal sensor (hottest temperature + bearing), floor sensor (mm), encoders + gyro (dead reckoning with noise and bias), battery level, tilt (always 0 in the sim).	[IMPLEMENTED IN SIM]

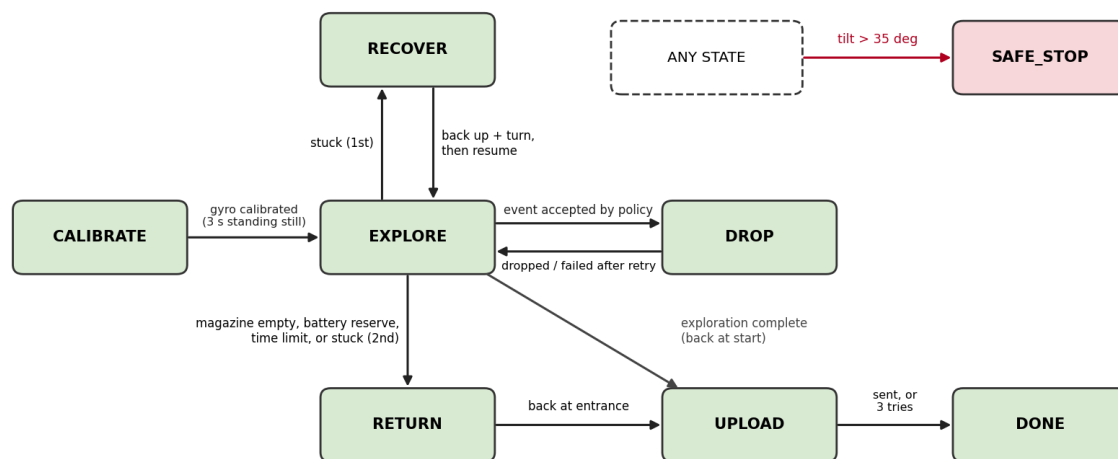
4.2 Responsibilities

- Explore a GPS-denied, unmapped space alone and autonomously.
- Know its own position well enough (dead reckoning + wall-based heading correction).
- Detect events, decide which are worth a beacon, and drop beacons within a limited stock.
- Record everything in a log, return to the entrance, and upload the log to the Outside Network.
- Survive faults safely (stuck, drop failure, low battery, tip-over).

4.3 Autonomous exploration and navigation strategy **[IMPLEMENTED IN SIM]**

- **Right-hand wall following** with LiDAR: keep ~0.40 m from the right wall; turn left in place if blocked ahead (<0.5 m) or if the floor is missing; curve right when the right wall disappears (opening). The controller uses the lateral error plus the wall angle (from a line fit of right-side LiDAR points, used only if the fit is good) and slows down in turns.
- **Localization**: dead reckoning (encoders for distance, gyro for heading, gyro bias calibrated during 3 s standing still) plus a **heading correction from walls**: when a straight wall is seen, the heading is nudged towards the nearest multiple of 90°. **[ASSUMPTION]** This assumes walls are parallel to the x/y axes (see section 6).
- **Breadcrumb trail with loop erasure**: positions are stored every 0.25 m; if the robot returns to an earlier place, the loop in between is deleted, so the trail is always the shortest known way back (dead ends leave no trace).
- The right-hand rule covers the whole building only if the building has no islands/loops (the simulated building is tree-like) **[ASSUMPTION]**.

4.4 State machine [IMPLEMENTED IN SIM]



EXPLORE contains: wall following, event detection, beacon decision, breadcrumb trail. UPLOAD sends the log only if the robot is near the entrance. RETURN retraces the loop-erased breadcrumb trail.

Figure 2 — Writer state machine as coded in `writer.py`. EXPLORE does the wall following, event detection and beacon decision. RETURN is used for early returns; when the right-hand tour ends back at the start, the robot goes directly to UPLOAD.

4.5 Event detection [IMPLEMENTED IN SIM] · event set [PROPOSED]

The challenge needs **at least 2 event types**; the current design has three, all listed as valid examples in slide 1 (hazard, hole, junction). The event set has not been formally confirmed by the user.

Event	How it is detected	Notes
HAZARD (heat)	Thermal reading ≥ 45 °C. The Writer follows the reading up to its peak and emits one event. Severity = $(\text{peak} - 45) / (85 - 45)$, clipped to 0..1. Range to the source is estimated from the peak with an assumed linear model (approximate).	Needs a real thermal sensor (not yet chosen). Demo with a safe heat source such as a heat pad — never real fire.
JUNCTION	From the LiDAR: at least 3 of 4 directions (front, left, right, back) are “open” for 2 consecutive scans. Open = free range > 1.5 m (right side: > 1.0 m, the same test the wall follower uses). Records a 4-bit <i>exits</i> mask.	Needs no extra hardware. In an early run a junction fired at a corridor-to-room doorway (the merge radius was widened afterwards; not re-checked). False junctions inside rooms have not been examined.
HOLE (missing floor)	The down-facing distance sensor, 0.30 m ahead, reads > 150 mm instead of ~60 mm. The robot stops and turns away (it never drives into it).	Uses the Modulino Distance in the proposed role. A hole that spans the corridor acts as a blocked path.

4.6 Beacon decision policy [IMPLEMENTED IN SIM] [ASSUMPTION]

Each candidate event gets a **score**; it is dropped only if the score is above a **threshold that rises as the magazine empties**, so the last beacons are kept for the most important events.

- Score: HAZARD = $0.6 + 0.4 \times \text{severity}$ · HOLE = 0.9 · JUNCTION = 0.5 · WAYPOINT (relay beacon when the gap to the last beacon exceeds the radio spacing) = 0.3 , rising up to 0.7 as the gap doubles.
- Threshold = $0.2 + 0.7 \times (1 - \text{stock} / \text{capacity})$. With capacity 8: 0.29 with 7 left, 0.55 with 4 left, 0.81 with 1 left.
- No new beacon of the same type within a “novelty radius” of an existing one (HAZARD 2.0 m, HOLE 1.5 m, JUNCTION 2.0 m, WAYPOINT 2.0 m); no beacon within 0.6 m of any other (events merge).
- Every decision (drop / skip / fail) is logged with its reason, so the behaviour can be explained in the report. Trade-off visible here: when stock is low, chain continuity (relay beacons) can lose against event priority.

4.7 Beacon deposition [IMPLEMENTED IN SIM] (mechanism [DESIGN ONLY])

On an accepted event the robot stops and builds the message (id, type, dir_deg, dist_m, t, ver [, exits]). The HAL call `drop_beacon(message)` stands for: write the message into the beacon by radio → servo releases it → break-beam confirms the fall. The robot waits 1 s; if the drop failed it retries once; if it fails again it logs “fail” and the beacon stays in the magazine. In the simulation the beacon lands at the robot’s true position. No radio propagation is modelled.

4.8 Logging, return to the entrance, upload [IMPLEMENTED IN SIM]

- **Log:** path (x, y, heading, time every 0.5 s), beacons, all decisions with reasons, statistics (path length, heading corrections, drop failures, stock left, battery, stuck events).
- **Return triggers:** magazine empty · battery $\leq 30\%$ · time limit 900 s · stuck twice · “exploration complete” (back near the start after having gone > 3 m away).
- **Return behaviour:** follow the loop-erased breadcrumb trail backwards (pure pursuit to a point ≥ 0.6 m ahead) with a front-obstacle and no-floor safety.
- **Upload:** the log is generated and sent through the HAL; in the sim it only succeeds if the robot is within 1.5 m of the entrance; up to 3 tries, otherwise the status is “upload_failed” and the log stays on board.

4.9 Confirmed versus assumed (Writer)

Confirmed (official documents / user)	Assumed or proposed (to validate)
Challenge rules and the Writer’s role · user will use UNO Q (“Q”), RPLIDAR, servo dropper · user owns Modulino Movement and Modulino Distance	Exact board model and LiDAR model · environment · event set · Modulino Distance as hole detector · thermal sensor choice · beacon write method · drive base, encoders, motors, power · all numbers in section 6.3 · wall-based heading correction · tree-like building · Brain/Reflex software split on the UNO Q

5. Current simulation

A Python simulation of the Writer only. The Writer’s decision code talks to the world **only through a Hardware Abstraction Layer (HAL)**, so the same logic could later run on the real robot by writing a “real” HAL. [IMPLEMENTED IN SIM]

File	Role
hal.py	The contract: abstract class <i>WriterHAL</i> with the functions the Writer may use — time (now, step), drive(v, w), get_pose, correct_heading, measured_speed, calibrate_imu, tilt_deg, battery, scan (LiDAR), read_thermal, floor_distance_mm, drop_beacon, upload_log. Documents the conventions (private frame, angles, units) and which UNO Q side would serve each call in Phase 2.
sim_world.py	<i>World</i> : a 14 m × 8 m tree-like building on a 0.25 m grid — main corridor (1 m wide), four branches A-D, three rooms R1-R3, two heat sources, one collapsed-floor hole, optional low debris (invisible to the LiDAR). <i>SimHAL</i> : the simulated robot and sensors with imperfections (gyro bias 0.3°/s, encoder scale error 1.5 %, LiDAR noise, wheel slip); the Writer sees only its own <i>estimated</i> pose, never the true one.
writer.py	The Writer’s brain: <i>WriterConfig</i> (all tunable numbers), <i>State</i> (the state machine), <i>Writer</i> (wall following, heading correction, breadcrumb trail, event detection, beacon policy, drop sequence, return, upload, log export).
run_demo.py	Command-line runner. Options: --seed --stock --drop-fail-prob --battery-drain --kill-at --debris --out --gif. Prints a result summary and decisions, writes <i>writer_log.json</i> (if uploaded), <i>beacons_on_ground.json</i> and a PNG (and optionally a GIF).
viz.py	Plots: map, true path, the Writer’s own estimate, beacons by type; optional animated GIF. (The PNG output was produced in earlier runs; the GIF function has never been run .)
test_robust.py	Helper created during debugging: runs many seeds and prints a one-line verdict each. Its “clean run” criterion is lenient (it does not check coverage of the building).

Run: `python run_demo.py --out out` (Python 3 with numpy, matplotlib and Pillow; versions were not recorded). The folders `out_default`, `out_s3`, `out_s10` in the working directory are **stale intermediate outputs and must not be used as results**; they are not in the snapshot.

5.1 What the simulation currently demonstrates (in normal runs)

- The Writer explores the simulated building autonomously, using only noisy LiDAR, thermal, floor, encoder and gyro data.
- It detects HAZARD, JUNCTION and HOLE events, scores them, and decides whether to deposit a beacon according to the policy.

- It simulates beacon deposition (beacons are placed on the ground with their messages) and records every decision with a reason.
- It generates a log (I3 draft), returns to the entrance, and exports/uploads the log.
- Typical normal run in the best seeds: about 57 m of path, about 250 s of simulated time, 5 beacons (2 HAZARD, about 2 JUNCTION, 1 HOLE). Earlier runs showed an end-of-run position error of 0.02–0.08 m between the true and estimated position.

[ASSUMPTION] No physical validation of any kind has been done. All numbers come from a simulation with invented sensor models.

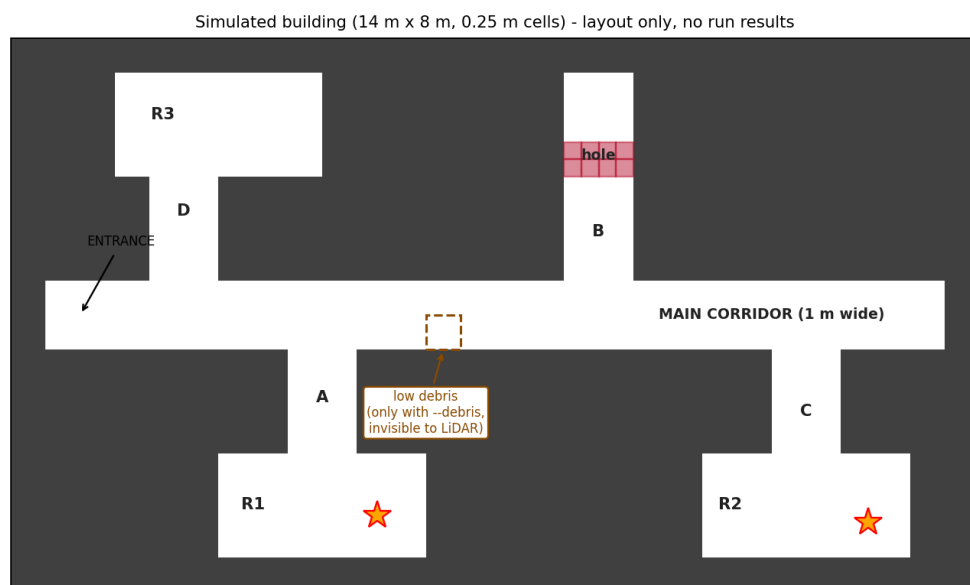


Figure 3 — Layout of the simulated building (static drawing, no run results). The robot starts at the entrance heading east.

6. Simulation limitations and debugging status

Read this section carefully. The simulation is **not yet fully reliable**. It works in many runs but is not consistent enough to be presented as a clean demo. The remaining problems have **not been diagnosed**.

6.1 What the multi-seed tests found **[KNOWN BUG]**

Normal scenario only (no failure options). A first check on seeds 0–11 found that all 12 runs ended “returned” with HAZARD, JUNCTION and HOLE beacons — but that check does not look at coverage. A second check on seeds 0–13 compared the true path to five zones (rooms R1, R2, R3, the hole corridor, the east end of the main corridor):

Observation	Seeds	Comment
A whole room was skipped	seed 3 (room R1, only 1 hazard found); seed 10 (room R3)	Real problem; the right-hand tour did not cover the building.
Wrong “stuck” event in a world without debris	seed 10 (1 stuck event)	Real problem; the robot was blocked by something the controller should have avoided (probably a wall collision).
Path much longer than normal	seed 1: 110.0 m, 479 s · seed 9: 81.3 m, 349 s (normal $\approx$ 46–58 m, $\approx$ 210–250 s)	Probably re-circling or wandering; cause unknown.
East end “missed”	seeds 0 and 7	Probably a test artefact (the test zone $x \geq 12.8$ m is tighter than the robot’s stop distance before the wall). Not verified.
Possible duplicate hazard beacon	seed 4 (3 HAZARD beacons for 2 heat sources)	Not examined; may be a split detection of one hot spot.
Seeds without any flagged problem	2, 4 (see above), 5, 6, 8, 11, 12, 13	Most runs are fine, but the system is chaotic: small noise changes can change the whole trajectory.

[KNOWN BUG] The right-hand exploration behaviour has a bug that is being investigated. The plots for seeds 3 and 10 were generated but **not yet inspected**; that was the immediate next step when

development was stopped. Next-agent tip: instrument the Writer from the inside (a logging hook in writer.py). Calling `hal.scan()` from a debug script consumes the random-number stream and changes the trajectory (this misled one debugging session).

6.2 Bugs already found and fixed in this session (effect not individually re-verified)

- Junctions on the **right** side were missed because the robot turns into them before they are confirmed → junction test for the right side now uses the same threshold as the wall follower; confirmation shortened to 2 scans; junction merge radius widened to 2.0 m.
- The LiDAR line-fit was polluted near openings and made the robot steer early into branches → the fit is now used for control only if it is good (residual < 0.05 m, length ≥ 0.7 m).
- The controller ignored wall angles above 45°, which caused a wrong left turn (and a U-turn back to the entrance) when leaving branch A → large angles are accepted (clipped), the robot slows in turns, gains retuned (kd 1.2, kphi 2.0).

6.3 Simplifications and assumptions [ASSUMPTION]

- Heading correction assumes walls are aligned with the x/y axes** (Manhattan-world). It stands in for real LiDAR scan matching and would be wrong in caves, mines with curved tunnels, or rotated buildings. The report must say so.
- The building is tree-like (no loops/islands) so the right-hand rule can cover it; a building with loops would break this.
- No radio model: beacon range, reception, interference and aging are **not simulated**. The beacon spacing is only a policy number.
- Not simulated at all: Outside Network, frame translation, Executor, command-post live map, trust-by-age behaviour. Beacon aging is not implemented in the Writer code (it belongs to the Beacon/Executor side).
- The sensors are simplistic (e.g. thermal field of view ±70° with linear falloff; floor sensor 60/240 mm). Tilt is always 0, so the tip-over safety can never trigger in the sim.

Parameters that are NOT validated (all [ASSUMPTION]; most can only be validated on real hardware):

Group	Parameter = value (as in the code)
Motion	cruise speed 0.30 m/s · wall distance 0.40 m · front stop 0.50 m · “wall lost” threshold 1.0 m · gains kd 1.2, kphi 2.0 · turn rate 1.0 rad/s · gyro calibration 3 s
Sensors / events	heat threshold 45 °C · severity reference 85 °C · assumed source temperature for range 80 °C · hole threshold 150 mm · floor look-ahead 0.30 m · “open direction” 1.5 m · junction confirmation 2 scans
Beacon policy	magazine capacity 8 (the real number of beacons is unknown) · radio spacing 3.5 m (must be measured) · min gap 0.6 m · novelty radii 2.0/1.5/2.0/2.0 m · scores 0.6+0.4·sev / 0.9 / 0.5 / 0.3→0.7 · threshold 0.2+0.7·(1–stock/capacity) · drop wait 1 s · 2 attempts · confidence = max(0.2, 1 – path/200 m) (heuristic)
Return / safety	battery reserve 30 % · time limit 900 s · loop-erase radius 0.32 m · stuck = speed < 0.03 m/s for 2 s while commanded > 0.1 m/s, 2 events → return · tilt limit 35°
Trust / aging thresholds	Slide examples: 2 / 15 / 40 min. AI-proposed: fresh < 5 min, aging 5–30 min, stale > 30 min, suspect = contradicted. Not implemented in code; [OPEN] (Beacon + Executor teammates).
Simulated physics	gyro bias 0.3°/s (0.03°/s left after calibration) · encoder scale error 1.5 % · LiDAR 120 rays, 6 m, noise 0.02 m · robot radius 0.14 m · thermal source 90 °C, linear falloff to 25 °C over 2.5 m, range 3 m · battery drain constant

7. Open design decisions

“Responsible teammate” assumes the four-role split proposed by the AI (Writer = the user). Names and the actual allocation of the other three people are unknown. **No decision below has been made silently; all need a human decision.**

Item	Current status	Decision still needed	Responsible teammate
Environment	Fire / Hazardous Building [PROPOSED] (sim built for it)	Confirm, or switch to another of the four environments	Whole team (user decides)
Event types	HAZARD, JUNCTION, HOLE coded. Earlier default discussed: hazard + junction	Confirm the set; every type must also be understood by the Executor and shown on the live map	Writer owner + Executor teammate
Distance sensor role	Assumed floor-facing hole detector; the user never answered the question	Choose: hole detector, drop confirmation, or front safety	Writer owner (user)

Item	Current status	Decision still needed	Responsible teammate
Beacon write method ("D3")	Proposed: blank beacon with unique ID, message written by radio at drop time. Alternatives: wired contacts in the magazine; pre-programmed beacons	Pick one; it fixes interface I1 and the Writer's radio hardware	Beacon teammate + Writer owner
Beacon message format	Fields proposed (section 8); byte layout, encoding, CRC and RF technology (BLE / ESP-NOW / LoRa) not defined	Define bytes, RF technology, range test plan	Beacon teammate
Beacon-to-beacon direction	[OPEN DESIGN ISSUE] see section 8.2	Decide how the chain is represented and who builds it	Beacon + Outside Network + Executor
Outside Network log format (I3)	Draft JSON produced by the sim (section 8.3)	Confirm fields, units, frame, status values	Outside Network teammate
Outside Network design	Raspberry Pi + GPS + MQTT + web map; satellite link simulated [PROPOSED]	Confirm architecture and what is simulated in Phase 1	Outside Network teammate
Frame translation inputs	Entrance GPS fix + building orientation [PROPOSED]	How to obtain the orientation; error model	Outside Network teammate
Shared frame with the Executor	Same private frame idea; heading consistency not solved	How the Executor knows its heading inside	Executor + Outside Network
Aging / trust thresholds	Slide examples vs AI proposal; not coded	Choose thresholds and who applies them (beacon message vs Executor)	Beacon + Executor teammates
Radio range and spacing	3.5 m [ASSUMPTION]	Measure on hardware; set spacing	Beacon + Writer
Clock synchronisation	GPS time before entry [ASSUMPTION]	Who sets the clocks and how	Outside Network teammate
Phase 1 submission details	Not in the documents; a Google Forms link exists in the PDF	Open the links; email aess@ieee.tn / ras@ieee.tn if unclear	User
Deadline and scoring grid	Deadline 05/10/2026 vs "due today"; 45 vs 60 points	Verify with organizers	User

8. Beacon information

8.1 Current proposed beacon message **[PROPOSED]** **[IMPLEMENTED IN SIM]**

Field	Meaning (current design only)	Maps to slide item
id	Unique beacon number given by the Writer, 1, 2, 3 ... (increases only after a successful drop).	identifier
type	What the beacon is about: HAZARD, JUNCTION, HOLE or WAYPOINT (a relay beacon dropped only to keep the chain unbroken). In the sim it is a text label; the Beacon teammate must encode it in a few bits.	WHAT
dir_deg	Direction 0–359° in the private frame (0 = direction of entry, counter-clockwise positive). Depends on type: HAZARD = bearing towards the hot spot; HOLE = the Writer's heading when it stopped (towards the missing floor); JUNCTION and WAYPOINT = the Writer's travel heading at the drop.	WHERE
dist_m	HAZARD = <i>rough</i> estimated range to the heat source (from the peak temperature with an assumed model); HOLE = 0.3 m (floor-sensor look-ahead); JUNCTION and WAYPOINT = the nominal radio spacing 3.5 m, a placeholder, not a real distance to the next beacon.	distance
t	Time the message was written (epoch seconds). Needs a GPS-synchronised clock [ASSUMPTION] . Used for aging by whoever reads the beacon.	WHEN
ver	Version, 1 at drop; a later agent can overwrite with a higher number. Not used anywhere in the code yet.	version
exits (junctions only)	4-bit mask of open directions in the private frame, rounded to the nearest 90°: bit0 = East (0°), bit1 = North (90°), bit2 = West (180°), bit3 = South (270°). Example 0111 = open East, North and West. Computed from the Writer's heading and the LiDAR.	— (extra; not on the slide)

Byte layout, number of bits per field, CRC and the radio technology are **not defined**. (An earlier chat estimate of "about 11 bytes" was only a rough guess **[ASSUMPTION]**.) The log also stores per beacon, *not on the air*: position x, y in the private frame, severity, decision score, confidence.

8.2 [OPEN DESIGN ISSUE] — the next beacon is unknown at drop time

The Writer does not know where the next beacon will be when it drops the current one. The slides describe a beacon as holding “direction to take from here” and “metres to next beacon / target”, but at drop time that next beacon does not exist yet. Therefore, in the current concept each beacon stores information about the **detected event** (type, direction and rough distance to the event), and the placeholder distance for junction/relay beacons is only the nominal spacing. The **beacon-to-beacon chain** (which beacon follows which, with the true direction and distance between them) may have to be **constructed later by the Outside Network** from the Writer’s log and delivered in the Executor’s briefing.

This needs agreement with the Beacon and Outside Network teammates (and the Executor teammate). Options for discussion — none is decided: (a) as now: event info on the beacon, chain built by the Outside Network; (b) the Writer re-writes the previous beacon after dropping the next one (hard mechanically); (c) each beacon stores the direction/distance *back* to the previous beacon, which is known at drop time; (d) a combination. Option (a) is what the code does today and it makes the Executor depend on its briefing rather than only on the beacons — a point judges may question.

8.3 Log sent to the Outside Network (interface I3, draft) [DRAFT] [IMPLEMENTED IN SIM]

JSON, all positions in the private frame (illustrative structure, not a result):

```
{ "robot": "writer", "frame": "private", "origin": "entrance", "axes": "x = direction of entry, y = left, angles CCW (degrees)",
  "start_time": <epoch s>, "end_time": <epoch s>, "status": "returned", "return_reason": "...",
  "path": [[x, y, heading_rad, t], ...],
  "beacons": [{"id", "type", "dir_deg", "dist_m", "t", "ver", ["exits"], "x", "y", "severity", "score", "confidence"}, ...],
  "decisions": [{"t", "type", "action": "drop|skip|fail", "score", "reason", "x", "y"}, ...],
  "stats": {"path_length_m", "heading_corrections", "drop_failures", "stock_left", "battery_end", "stuck_events"} }
```

If the Writer is lost, no log exists; the sim then writes only *beacons_on_ground.json* (what an Executor could still read).

9. Failure cases

Important: only normal runs have been executed. **None of the failure scenarios below has been run.** The first five exist as code paths or command-line options; the others are design ideas only.

Scenario	What fails	What the simulation should demonstrate / expected response	Status
Writer dies --kill-at T	Power/crash at time T, or falling into the hole	No log is uploaded (status “lost”); beacons already dropped stay on the ground and are written to <i>beacons_on_ground.json</i> — the core “knowledge survives” story.	[UNTESTED]
Servo / drop failure --drop-fail-prob p	Servo jam or radio write fails; no break-beam confirmation	Retry once; if it fails again log “fail”, keep the beacon in the magazine, continue exploring.	[UNTESTED]
Low battery --battery-drain x	Battery reaches the 30 % reserve	Stop exploring, return by the breadcrumb trail, upload the log (reason “battery reserve reached”).	[UNTESTED]
Hidden debris / stuck --debris	Low obstacle the LiDAR cannot see blocks the wheels	Stall detected (commanded > 0.1 m/s, measured < 0.03 m/s for 2 s) → back up and turn; a second stall → return with reason “stuck”. (A stuck event also appeared <i>without</i> debris in seed 10 — a bug symptom, see 6.1.)	[UNTESTED]
Magazine empty / time limit	No beacons left / mission too long	Return and upload (coded in the return triggers).	[UNTESTED]
Upload failure	Robot not within range of the Outside Network	Up to 3 tries, then status “upload_failed”, log kept on board (coded).	[UNTESTED]
Tip-over	Robot flips or tilts > 35°	SAFE_STOP (coded) — cannot be triggered in the sim because tilt is always 0.	[UNTESTED]
LiDAR dropout	No scans arrive	Stop, then use the breadcrumb return.	[DESIGN ONLY]
“Brain” crash	Linux side stops sending commands	Microcontroller watchdog stops the motors (Phase 2 hardware idea).	[DESIGN ONLY]
Localization drift	Gyro/encoder errors grow	Wall-based heading correction in the sim; confidence value on each beacon. No dedicated failure test.	Partly modelled; [UNTESTED] as a scenario

Scenario	What fails	What the simulation should demonstrate / expected response	Status
Stale or contradicted beacon; missing beacon; Outside link loss	Executor / Outside Network side	Trust-by-age rules; fallback to own sensors; store-and-forward queue.	[DESIGN ONLY] (other teammates; no code)

10. GitHub repository plan

[PROPOSED] No repository exists yet as far as this project record shows. Goal: a judge should understand the project in two minutes.

```
living-map/
├── README.md
├── requirements.txt
├── simulation/
├── docs/
├── results/
└── report/
```

Path	What goes in it
README.md	One-paragraph pitch · the challenge in 5 lines · architecture figure · how to run the demo (exact commands) · what is simulated and what is not · results figure · links to docs/ and report/ · team and roles · honest limitations.
requirements.txt	numpy, matplotlib, Pillow (the libraries used so far; add versions once known).
simulation/	hal.py, sim_world.py, writer.py, run_demo.py, viz.py, test_robust.py; later the Outside Network, frame translation and Executor modules written by the teammates; a short README of its own with the command-line options.
docs/	Architecture and data-flow diagrams · Writer state machine · beacon message format · interface specs I1-I5 · frame-translation note · an ASSUMPTIONS list (section 6.3) · this handover PDF · implementation plan.
results/	Only outputs generated from the final code: normal-run PNG/GIF, sample writer_log.json and beacons_on_ground.json, and one output per failure scenario with a caption saying what it shows. No stale files.
report/	Final technical report PDF (max 6 pages) and its source.

11. Technical report plan (maximum 6 pages)

The report itself is worth only 2 points, but it carries the explanation of the seven technical areas (35 points), the diagrams (5), the plan (3) and the failure cases (3). Page budgets are a **[PROPOSED]** guide (total ≈ 5.8 pages).

#	Section	Pages	Available now?	Still missing
1	Introduction and Challenge	0.4	Material available (section 1 here)	Write it for the report
2	Proposed Solution	0.5	Material available (section 3)	Confirm environment/events; innovation paragraph
3	System Architecture	0.7	Draft diagram (Figure 1)	Final diagram + data-flow diagram; confirm interfaces with teammates
4	Writer Robot	0.7	Material available (section 4); state machine drawn	Write-up; hardware wiring sketch
5	Event Detection and Beacon Deposition	0.6	Material available (4.5–4.7)	Clean simulation results after the bug fix
6	Beacon Message and Communication	0.5	Partial (section 8)	OPEN: byte layout, RF technology, chain issue, aging rules
7	Outside Network and Executor	0.7	Partial — concept only	Teammates' designs; frame-translation example with numbers; Executor logic
8	Simulation	0.5	Partial (section 5)	Reliable run, figures, demo video
9	Failure Cases	0.4	Partial (section 9)	Run and record the scenarios
10	Implementation Plan	0.5	Partial (hardware list, Phase 2 items)	Timeline to 01/12/2026, budget, risks, owners
11	Limitations and Future Work	0.3	Material available (section 6, 12)	Write it for the report

12. Work remaining

MUST DO FOR PHASE 1

	Task	Why / note
☐	Verify the real deadline and submission route: open the two links in the cahier des charges (architecture diagram, Google Form); email aess@ieee.tn / ras@ieee.tn if unclear.	Deadline is 05/10/2026 in the document.
☐	Diagnose and fix the exploration bug (skipped rooms, false “stuck”, wandering). Inspect the plots of seeds 3, 10, 1, 9 first; instrument from inside the Writer.	Then rerun a multi-seed test with a proper coverage criterion until results are consistent.
☐	Run and record the failure scenarios (writer dies, servo jam, low battery, debris, magazine empty, upload failure).	Needed for “Failure Cases” and part of the demo.
☐	Generate clean simulation outputs (PNG, GIF, logs) from the final code; delete stale outputs; test the GIF function once.	Needed for the demo and the report.
☐	Minimal simulation of the rest of the chain: Outside Network (receive, frame translation to GPS, forward, simple live map) and Executor following beacons in the same world.	Today only the Writer is simulated, but the 7 required areas include them. Assign owners; agree the interfaces first.
☐	Agree interfaces with teammates: beacon message/D3, log format I3, direction meaning (section 8.2), event set.	Section 7.
☐	Diagrams: final system architecture, data flow, Writer state machine (draft exists), beacon message layout, frame translation.	5 points for diagrams.
☐	Implementation plan: hardware list, timeline to 01/12/2026, risks, who does what.	3 points.
☐	Technical report (≤ 6 pages) following section 11.	2 points + carries the rest.
☐	README and GitHub repository organised as in section 10.	4 points.
☐	Final check against the six deliverables and the seven areas.	Last step before submitting.

SHOULD DO IF TIME ALLOWS (realistic only)

- Add a simple radio-range model to the simulation so the beacon spacing is justified, not just assumed.
- Implement beacon aging/trust in the simulated Executor (Beacon/Executor teammates).
- Add a second building layout containing a loop, to show honestly where the right-hand rule fails.
- Compare the wall-based heading correction against a simple scan-matching correction.
- An innovation paragraph for the report: score-based beacon rationing, aging and trust, the dual-processor split of the UNO Q.
- Pitch outline (the pitch is only in the final phase).

FINAL PHASE / LATER (physical work — nothing here has been started)

- RPLIDAR integration on the UNO Q (including the powered USB-C hub question) and real scan processing.
- Real beacon radio tests: technology choice, range inside a building, power use.
- Physical beacon dispenser (servo gate, magazine, break-beam).
- Drive base, encoders, motor driver, power rails, e-stop; thermal sensor; Modulino Movement/Distance integration.
- Real robot navigation and tuning of every parameter in section 6.3; hardware validation of the Writer, beacons, Executor.
- Real Outside Network (GPS + link to a command post); complete physical prototype (15 points); user manual; pitch in English (5 min + 2 min Q&A).

13. Instructions for the next AI agent

- **Read this PDF first**, completely, then **read the official cahier des charges** (The_Living_Map_Spatial_Memory_for_Emergency_Robots_1.pdf) and the presentation (presentation.pdf). They are the primary source of truth. Never invent requirements.
- **Do not restart the project from zero** and do not redesign decided parts without a reason. Decided: the challenge reading, the four-role split, the Writer hardware (UNO Q, RPLIDAR, Modulino Movement and Distance, servo), the HAL-based simulation, the beacon policy structure.
- **Continue from the current status** (section 14). Ask the user for the zip writer_sim_snapshot_2026-10-03.zip; the working folder of the previous session may no longer exist.
- **First finish the missing Phase 1 deliverables**, in this order: verify the deadline and submission route → fix/stabilise the simulation → run the failure scenarios → clean results → diagrams → report → README/GitHub → final check (section 12).
- **Keep assumptions and untested items clearly labeled** with the labels of this document. Never claim physical validation. Never present an untested scenario as tested.
- **Priorities:** working simulation, report, diagrams, GitHub, failure cases and implementation plan — in that spirit, not new features.
- **Help the user step by step.** The user is new to the project. Use clear, direct language; explain technical words simply; do not dump large amounts of information; keep the pace fast because of the deadline.
- **Working agreement set by the user:** for each major decision give *Problem* → *Options* → *Trade-offs* → *Decision* → *Next action*; do **not** make the final choice silently; challenge assumptions; point out weak points and contradictions; distinguish what is technically possible now from what needs research; label assumptions; do not make unrealistic claims.
- **Known traps:** (1) the multi-seed “clean run” test is lenient — use coverage; (2) debugging scripts that call `hal.scan()` change the random stream; (3) the folders `out_default/out_s3/out_s10` are stale; (4) the heading correction only works for axis-aligned walls; (5) the scoring grid says 45 but sums to 60.

14. Final summary table

Area	Status	Priority
Challenge understanding	Done	Low
Writer concept	Done	Low
Writer simulation	Mostly done / debugging (not reliable yet)	HIGH
Event detection	Designed (implemented in sim)	HIGH
Beacon logic	Designed (implemented in sim)	HIGH
Beacon interface	Open	HIGH
Outside Network interface	Draft	HIGH
Failure cases	Designed, testing needed	HIGH
GitHub	Not packaged yet	HIGH
Diagrams	Not completed (two drafts in this PDF)	HIGH
Report	Not completed	HIGH
Implementation plan	Partial	HIGH
Physical hardware validation	Not done	Later
<i>Added by the AI for completeness:</i> Outside Network + Executor simulation	Not started	HIGH
<i>Added:</i> Deadline / submission route / scoring clarification	Open (links not yet opened)	HIGH

Appendix — How the project got here (short timeline)

Step	What happened
1	The two official documents were read; the challenge, Phase 1 deliverables, problems and three solution levels were analysed.
2	The team split into four roles was proposed (Writer, Beacon, Outside Network, Executor) with a shared interface contract I1-I5.
3	The user, owner of the Writer, chose UNO Q + RPLIDAR + servo; the AI filled the gaps (drive base, power, thermal sensor, beacon write method...). The IMU was explained; the user then said they own Modulino Movement and Modulino Distance.
4	The AI built the Writer simulation (hal, sim_world, writer, run_demo, viz, test_robust). First runs worked; several bugs were found and fixed (section 6.2).
5	Multi-seed testing exposed remaining problems (section 6.1). Development was stopped on purpose to write this handover.